

Our Plan for testing our front end will involve having dedicated directories for each test case, for example we will have login_output_dir, logout_output_dir etc. We will also have a directory containing shell scripts for each test case and its functions.

Github: <https://github.com/OwaisAhms/CSCI-3060-course-project>

Login:

For the login test case we will be testing the user inputting their username and password into the command line, the main features we will be testing first would be whether the system correctly checks the Current User Account file for if the username is valid or not i.e if the user even exists in the first place as well as if the password is valid, on successful login the command line will display the users information in the format as seen in the daily transaction file with a message displaying a successful login example: User001_____ AA 999999.99 logged in.

Logout:

For the logout test case the user will be prompted with a message displaying the current user and to type 'logout' if they wish to logout, we will be testing if the system is correctly tracking the current state of the user, if they are logged in or not, the user must only be able to logout if they are logged in. Upon logout we must also test if the daily transaction file has also been properly updated with that user's transactions for the day.

Create:

For the create test case we will test if the user attempting to use this function is an admin or not, only users logged in as an admin will be able to create new accounts. Next we will also have to test if the user is properly added to the current users account file.

Delete:

For the delete test case we will test to see if the user is an admin then they would be able to remove a user. The user must be an existing user but not the current user and deleted users afterwards cannot do nothing.

Sell:

For the sell test case we will test to see if the user is any type of account except standard__buy which will then allow them to put up a game for sale. The game must have a maximum length of 25 characters and a maximum price of \$999.99. The game name cannot be the same as another game.

Buy:

The buy test case is the ability to make purchases for games and have them saved into your collection. The buyer must have the price of the game or more of credit and not be standard_sell to make the purchase. The buyer must be debited the price of the game and the seller must be credited the amount.

Refund:

For the refund test case the user will be tested to see if they are an admin or not since it is a privileged transaction only allowed by the admin. Then the buyer's and user's username will be tested to see both exist in the Current User Accounts File. If either don't exist then the refund won't happen. For a valid refund we will test if the correct amount of credit is transferred from the seller's credit balance to the buyer's. Along with that we will test if the successfully refund amount is saved to the daily transaction file

Add credit:

For the addcredit function we will test it both in admin and standard user mode. After logging in the admin should be asked the amount of credit to add and the username of the user they are transferring to. We will test if the user having credit added has an existing username in the Current User Accounts file before adding the credit. When logged in as a standard user the system should only ask for the amount of credit to add to their own account. The maximum limit of adding credit in one session will be tested by testing amounts at and beyond the \$1000.00 limit. We will test if a successful add credit transaction is properly saved to the daily transaction file